

在這個範例中，我們使用了 **React Router v6.0** 版本中的 **BrowserRouter**、**Routes**、**Route**、**Link** 和 **Outlet** 元件來創建路由系統。**Link** 元件可以被用來建超連結，**Outlet** 元件可以被用來顯示子路由的內容。注意，在 **v6.0** 版本中，**Route** 元件的 **component** 屬性已被移除，改用 **element** 屬性來指定元件。

npm create-react-app react-router-dom-basic

npm install react-router-dom

```
import React from 'react';
import { BrowserRouter, Routes, Route, Link, Outlet } from 'react-router-dom';
```

```
const Home = () => (
  <div>
    <h2>Home</h2>
    <p>Welcome to my website!</p>
  </div>
);
```

```
const About = () => (
  <div>
    <h2>About</h2>
    <p>Learn more about me and my work.</p>
  </div>
);
```

```
const Contact = () => (
  <div>
    <h2>Contact</h2>
    <p>Get in touch with me.</p>
  </div>
);
```

```
const App = () => (
  <BrowserRouter>
    <nav>
      <ul>
        <li>
          <Link to="/">Home</Link>
        </li>
      </ul>
    </nav>
  </BrowserRouter>
);
```

```
    <li>
      <Link to="/about">About</Link>
    </li>
    <li>
      <Link to="/contact">Contact</Link>
    </li>
  </ul>
</nav>

<Routes>
  <Route path="/" element={<Outlet />}>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="/contact" element={<Contact />} />
  </Route>
</Routes>
</BrowserRouter>
);

export default App;
```

react-router-dom v6.0 的使用方式 Routes 元件取代 Switch 元件

在 react-router-dom v6.0 中，原本的 Switch 元件被取代成了 Routes 元件。Routes 元件的使用方式與 Switch 元件相似，用來根據路徑匹配對應的元件。

Route 元件仍然存在

在 react-router-dom v6.0 中，Route 元件仍然存在，但是使用方式有所變化。在新版本中，使用 Route 元件時需要在 element 屬性中傳遞對應的元件，而不再使用 component 屬性。以下是一個範例：

```
<Route path="/about" element={<About />} />
```

路由參數直接透過 props 傳遞

在 react-router-dom v6.0 中，路由參數不再透過 match.params 屬性取得，而是直接透過 props 傳遞到元件中。以下是一個範例：

```
function ProductDetails(props) {  
  const { id } = props;  
  // ...  
}
```

```
<Route path="/products/:id" element={<ProductDetails />} />
```

在這個範例中，當使用者訪問 /products/123 時，ProductDetails 元件會接收到一個名為 id 的屬性，其值為 123。

Link 元件仍然存在

在 react-router-dom v6.0 中，Link 元件仍然存在，並且使用方式與以前相同。例如，在上面的產品及購物車例子中，我們使用了 Link 元件來實現導航。以下是一個範例：

```
<Link to="/products">Products</Link>
```

這個 Link 元件會在點擊時將用戶導航到 /products 路徑。

總體來說，在 react-router-dom v6.0 中，路由的基本使用方式沒有太大的變化，只是一些屬性和元件名稱有所改變。新版本還引入了一些新功能，例如動態路由、路由守衛等等，可以在需要時使用這些功能來更好地管理應用程式的路由。

```

import { useState } from "react";
import { BrowserRouter, Routes, Route, Link, Outlet } from "react-router-dom";
import { useParams } from "react-router-dom";
function App() {
  const [cart, setCart] = useState([]);

  const products = [
    { id: 1, name: "Product 1", price: 100 },
    { id: 2, name: "Product 2", price: 200 },
    { id: 3, name: "Product 3", price: 300 },
  ];

  function Home() {
    return <h1>Welcome to the Home Page!</h1>;
  }

  function Products({ products }) {
    return (
      <div>
        <h1>Products</h1>
        <ul>
          {products.map((product) => (
            <li key={product.id}>
              <Link to={` /products/${product.id}`}>{product.name}</Link> -
              ${product.price}
            </li>
          ))}
        </ul>
        <Outlet />
      </div>
    );
  }

  function ProductDetails({ addToCart }) {
    const { id } = useParams();
    const product = products.find((p) => p.id === parseInt(id));

    function handleAddToCart() {

```

```

        addToCart(product);
    }

    return (
        <div>
            <h1>{product.name}</h1>
            <p>${product.price}</p>
            <button onClick={handleAddToCart}>Add to Cart</button>
        </div>
    );
}

function Cart({ cart, removeFromCart }) {
    const total = cart.reduce((acc, cur) => acc + cur.price, 0);

    function handleRemoveFromCart(product) {
        removeFromCart(product);
    }

    return (
        <div>
            <h1>Cart</h1>
            <ul>
                {cart.map((product) => (
                    <li key={product.id}>
                        {product.name} - ${product.price}{" "}
                        <button onClick={() =>
handleRemoveFromCart(product)}>Remove</button>
                    </li>
                ))}
            </ul>
            <p>Total: ${total}</p>
        </div>
    );
}

function addToCart(product) {
    setCart((prevCart) => [...prevCart, product]);
}

```

```

    }

    function removeFromCart(product) {
      setCart((prevCart) => prevCart.filter((p) => p.id !== product.id));
    }

    return (
      <BrowserRouter>
        <nav>
          <ul>
            <li>
              <Link to="/">Home</Link>
            </li>
            <li>
              <Link to="/products">Products</Link>
            </li>
            <li>
              <Link to="/cart">Cart ({cart.length})</Link>
            </li>
          </ul>
        </nav>

        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/products" element={<Products products={products} />} />
          <Route path="/cart" element={<Cart cart={cart}
removeFromCart={removeFromCart} />} />
          <Route path="/products/:id" element={<ProductDetails
addToCart={addToCart} />} />
        </Routes>
      </BrowserRouter>
    );
  }
}

export default App;

```